

Justificación Frameworks

3_ Extensión de Frameworks

1) Respecto a la primera propuesta, **hotspots con herencia**:

- **Responda las siguientes preguntas:**

- a. **¿Qué debo hacer si aparece una nueva fuente de energía (por ejemplo, paneles solares con baterías)? ¿Cuántas y cuáles clases debo agregar en caso de querer todas las variantes de robots posibles para este nuevo tipo de fuente de energía?**

Respuesta: Agregar una clase abstracta para la batería, una clase abstracta por cada sistema de locomoción(3) y como hay 2 tipos de armas hay que agregar 2 clases concretas por cada tipo de locomoción(6).

- b. **¿Puedo cambiarle, a un robot existente, el sistema de armas sin tener que instanciar el robot de nuevo?**

Respuesta: No, tengo que instanciar un nuevo robot siempre que necesite hacer un cambio, ya que no se puede cambiar en tiempo de ejecución.

- c. **¿Dónde almacenaría usted el nivel de carga de la batería? ¿Qué implicaría eso si antes de disparar el láser hay que garantizar que la fuente de energía puede satisfacer el consumo del arma?**

Respuesta: El nivel de carga podría almacenarse como una variable de instancia directamente dentro de la clase abstracta robot.

2) Respecto a la sección **Hotspots por composición**:

- **Responda las siguientes preguntas:**

- a. **¿Qué debo hacer si aparece una nueva fuente de locomoción (por ejemplo, motor con ruedas con tracción 4x4)? ¿Cuántas y cuáles clases debo agregar en caso de querer todas las variantes de robots posibles para este nuevo tipo de sistema de locomoción?**

Respuesta: En este esquema el robot delega responsabilidades, lo único que tengo que hacer es agregar la clase “motorWheels4x4” que extienda de “LocomotionSystem”

- b. **¿Puedo cambiarle, a un robot existente, el sistema de armas sin tener que instanciar el robot de nuevo?**

Respuesta: Si, a través de la composición puedes cambiar los tipos en tiempo de ejecución.

- c. **¿Dónde almacenaría usted el nivel de carga de la batería? ¿Qué implicaría eso si antes de disparar el láser hay que garantizar que la fuente de energía puede satisfacer el consumo del arma?**

Respuesta: En la composición los componentes están aislados y no pueden acceder al estado del otro. Esto obliga a diseñar interfaces públicas para que el arma pueda interrogar el nivel de carga de la batería utilizando al robot como intermediario.

3) Explique las ventajas y desventajas de las dos formas de extensión del framework (herencia y composición).

Herencia (Hotspots vía Template Method)

Este enfoque se basa en "por diferencia", implementando los ganchos (*hooks*) mediante la creación de subclases. El patrón Template Method implementa estas plantillas con herencia y mensajes a *this*.

Ventajas

- **Alta visibilidad interna:** Al heredar, tenés acceso directo a las variables de instancia y a los métodos que heredás de la clase madre, lo que lo hace muy flexible a la hora de codificar la lógica interna del método.
- **Simplicidad inicial:** Es intuitivo y directo de implementar cuando hay pocas variaciones.

Desventajas

- **Explosión de la jerarquía:** Si el dominio tiene muchas dimensiones de cambio (ej: tipo de batería + tipo de movimiento + tipo de arma), la cantidad de subclases explota y el código se vuelve inmanejable.
- **Rigidez estática:** No podés cambiar el comportamiento del objeto (el Robot) en tiempo de ejecución (*run-time*).

Composición (Hotspots vía Delegación / Strategy)

En este diseño, los ganchos (*hooks*) se resuelven utilizando objetos separados y delegando la responsabilidad de la acción en ellos.

Ventajas

- **Flexibilidad dinámica:** Te permite cambiar los tipos en tiempo de ejecución simplemente reemplazando uno de sus componentes.
- **Evita la explosión de clases:** Las combinaciones se arman inyectando dependencias al instanciar, manteniendo un número de clases bajo y altamente reutilizable.

Desventajas

- **Pérdida de visibilidad interna:** Los componentes están aislados. No tenés acceso a las variables de instancia ni a los métodos privados del objeto principal (Robot) ni al de los otros componentes hermanos.

- **Dependencia de la interfaz:** Tu código queda dependiendo estrictamente de las opciones públicas que provea el framework o el componente. Requiere diseñar canales de comunicación más complejos (como pasar a `this` por parámetro).